# Book Ingestion Architecture Review

# Book Ingestion Architecture Review

## Summary

The project already has the right foundation for retrieval-augmented generation: Supabase stores content chunks with pgvector embeddings, the app retrieves relevant chunks at chat time, and the system prompt forces answers to stay grounded in retrieved context.

The book should be ingested as a first-class, canonical source with chapter/page metadata, clean extraction, and retrieval evaluation before production use.

## Handoff Brief

This is not a platform rewrite. The existing app is close. The highest-value work is to improve retrieval quality, add richer source metadata, and introduce a small evaluation loop.

Recommended priority order:

1. Add book/source metadata and full-text search fields to Supabase.
2. Replace the vector-only RPC with hybrid search using pgvector + Postgres full-text search + Reciprocal Rank Fusion.
3. Build a dedicated book ingestion script with PDF cleanup, chapter-aware chunking, Q&A preservation, contextual chunk headers, idempotent hashes, and rich metadata.
4. Update prompt/source formatting so book chunks cite chapter/section/page and the model gives concise synthesized guidance.
5. Add a small eval suite for retrieval quality, citation grounding, and answer-quality regressions.

Non-goals for the first implementation pass:

- Do not introduce Elasticsearch, Pinecone, Qdrant, or a new vector database.
- Do not adopt a full RAG framework unless a specific library proves it removes real project complexity.
- Do not ingest the watermarked PDF directly into production.
- Do not rely on prompt wording alone to solve weak retrieval.

## Current Architecture

The current retrieval path is:

```
Text sources
  -> chunk text
  -> create embeddings
  -> store rows in Supabase content_chunk
  -> query match_content_chunks RPC
  -> format retrieved context
  -> inject context into the chat system prompt
  -> stream the assistant response
```

Relevant files:

- `supabase/migrations/001_initial.sql` defines `content_chunk`, the `vector(1536)` embedding column, the pgvector index, and `match_content_chunks`.
- `scripts/ingest.ts` reads local text files, chunks them, embeds new chunks, and inserts rows into Supabase.
- `lib/rag/chunker.ts` provides the paragraph/sentence chunker.
- `lib/rag/retrieval.ts` embeds the user query and calls `match_content_chunks`.
- `app/(chat)/api/chat/route.ts` retrieves chunks for the latest user message and builds the model prompt.
- `lib/ai/prompts.ts` requires answers to use only retrieved context.

## PDF Observations

The book PDF is readable and structurally usable for extraction:

- Title: *Was It Something I Said?*
- Author: Alison Cheperdak
- Length: 320 pages
- Extracted text: about 89,000 words
- The PDF is tagged, unencrypted, and extracts cleanly with `pdftotext`.

The extraction includes repeated watermark/footer lines on every page, including page numbers and distribution warnings. Those lines should be removed before embedding because they add noise and can weaken retrieval quality.

The table of contents and major chapter headings survive extraction well enough to support chapter-aware ingestion.

## Recommended Data Model Changes

The current `content_chunk.source` enum only allows:

```
blog, substack, evie, instagram, knowledge_base
```

For the book, add either a new `book` source value or, preferably, normalize source metadata into a separate `content_source` table.

Recommended metadata per chunk:

- `source`: `book`
- `title`: `Was It Something I Said?`
- `book_title`
- `author`
- `chapter_number`
- `chapter_title`
- `section_title`
- `page_start`
- `page_end`
- `chunk_index`
- `content_version`
- `is_retrievable`
- `content_hash`

This metadata lets the app cite retrieved material cleanly, filter out front matter, and re-ingest future editions without collisions.

### Low-Risk Migration Path

For a first implementation, keep the existing `content_chunk` table and add fields rather than replacing the schema:

- Add `book` to the allowed `source` values.
- Add `metadata jsonb not null default '{}'`.
- Add `fts tsvector generated always as (to_tsvector('english', content)) stored`.
- Add a GIN index on `fts`.
- Consider replacing or supplementing the existing `ivfflat` vector index with HNSW if supported by the deployed Supabase pgvector version.

- Add analytics fields for retrieved chunk IDs and scores, either to `chat_analytics` or a new retrieval event table.

Suggested `metadata` shape for book chunks:

```
{
  "book_title": "Was It Something I Said?",
  "author": "Alison M. Cheperdak",
  "chapter_number": 3,
  "chapter_title": "Main Character Energy: Job Search Edition",
  "section_title": "Interview Follow-Up",
  "page_start": 72,
  "page_end": 74,
  "chunk_index": 12,
  "authority_tier": 1,
  "content_version": "watermarked-pdf-2025-12-19",
  "pipeline_version": "book-ingest-v1",
  "is_retrievable": true
}
```

Longer-term, a normalized `content_source` table would be cleaner, but it is not required for the first pass.

---

## Recommended Ingestion Strategy

Ingest the book as a first-class source rather than as generic `knowledge_base` .

Suggested pipeline:

```
PDF
  -> extract text
  -> remove watermarks, footers, and non-content front matter as needed
  -> detect chapters and sections
  -> preserve Q&A pairs
  -> chunk by semantic blocks
  -> embed chunks
  -> insert into Supabase with rich metadata
  -> test retrieval with representative etiquette questions
```

Chunking recommendations:

- Prefer chapter/section/Q&A-aware chunks over raw word windows.
- Keep each question with its answer.
- Target roughly 450-700 tokens per chunk.
- Use modest overlap, around 60-90 tokens.
- Avoid embedding table of contents, notes, acknowledgments, watermarks, and repeated footers unless there is a specific product reason.
- Include chapter and page metadata with every chunk.

### Book Ingestion Script Contract

Create a dedicated script rather than extending the existing general content script too far.

Recommended script:

```
scripts/ingest-book.ts
```

Expected responsibilities:

- Accept a PDF path and source/version metadata.
- Extract text to a temporary local file.
- Remove repeated watermark/footer lines, page labels, and extraction artifacts.
- Preserve page boundaries before cleaning so each chunk can receive `page_start` and `page_end`.
- Detect chapter headings, section headings, and Q&A blocks.
- Keep Q&A pairs together whenever possible.
- Generate a short contextual header for each chunk before embedding, such as: `From Was It Something I Said?, Chapter 4, discussing workplace communication and meeting etiquette.`
- Embed the contextualized chunk text, while storing the clean display text separately if needed.
- Dedupe by hash of normalized content plus source/version metadata.
- Insert in batches and be safe to rerun.

For extraction, the local inspection showed `pdftotext -layout` works well enough for this PDF. If future PDFs have tables, columns, or scanned pages, consider a stronger extraction/OCR step before chunking.

---

## Retrieval Improvements

Before relying heavily on the book corpus, improve retrieval quality:

1. Add hybrid search.

Use pgvector similarity plus Postgres full-text search. Etiquette questions often contain exact phrases, and vector-only search can miss precise wording.

2. Retrieve more candidates.

   The current retrieval default is six chunks with a `0.3` similarity threshold. Pull 10-15 candidates, optionally rerank, then send the best 5-8 chunks to the prompt.

3. Make retrieval conversation-aware.

   The current chat route embeds only the latest user message. Follow-ups like "what if it is my boss?" need recent conversation context or a query rewrite step.

4. Improve source formatting.

   Instead of only showing `title` and `source`, format retrieved book chunks with chapter, section, and page range.

5. Track retrieval performance.

   Store chunk IDs and similarity scores in analytics so weak answers can be traced back to weak retrieval.

## Hybrid Search Implementation Notes

Use the official Supabase hybrid-search pattern as the starting point:

- Run a full-text candidate CTE using `fts @@ websearch_to_tsquery(query_text)`.
- Run a semantic candidate CTE using `embedding <=> query_embedding`.
- Apply source, authority, and `is_retrievable` filters inside both CTEs.
- Fuse results with Reciprocal Rank Fusion.
- Return chunk metadata, rank/similarity details, and enough provenance for citation.

Suggested RPC signature:

```
match_content_chunks_hybrid(
  query_text text,
  query_embedding vector(1536),
  match_count int default 8,
  full_text_weight float default 1,
  semantic_weight float default 1,
  rrf_k int default 50,
  source_filter text[] default null
)
```

Then update `lib/rag/retrieval.ts` to call the hybrid RPC with both the raw user query and the query embedding.

The current retrieval embeds only the latest message. For follow-up questions, add a small query rewrite step that combines the latest user message with the recent conversation context before retrieval.

---

## Prompt Recommendations

The current strict grounded prompt is useful, but once the book is available it should include clearer source handling:

- Tell the model to synthesize guidance rather than over-quote source text.
- Tell the model to cite the book naturally when book chunks are used.
- Keep the existing instruction to admit when retrieved context is insufficient.
- Avoid forcing a book mention when the retrieved context comes from non-book sources.

Example source label:

```
[Source 1: Was It Something I Said?, Chapter 3, "Main Character Energy: Job Search Edition", pp. 72–74]
```

---

## Security Recommendation

The remote ingestion endpoint currently authenticates with the last 10 characters of the Supabase service role key. Replace that with a dedicated `INGEST_SECRET`, or remove the public ingest endpoint and run ingestion only from a local/admin script.

The service role key should never be reused, partially exposed, or used as an implied shared secret.

---

## Evaluation Plan

Add a small eval loop before tuning chunk sizes or reranking.

Start with `promptfoo` because it fits a TypeScript/Next.js repo and can test retrieval/generation behavior in CI. `Ragas` or `DeepEval` can be added later for deeper offline metrics.

Minimum eval set:

- 20-30 realistic etiquette questions.
- 5-10 follow-up questions that require conversation-aware retrieval.
- 5 source-conflict questions where the book should outrank lower-authority content.

- 5 insufficient-context questions where the assistant should gracefully decline or redirect.

Track separately:

- Retrieval: expected chunk IDs or expected chapter/section.
- Generation: grounded answer, no unsupported claims, concise synthesis.
- Citation: every cited source ID maps to an actually retrieved chunk.

Log at runtime:

- user question
- rewritten retrieval query, if used
- retrieved chunk IDs
- source/title/chapter/page metadata
- similarity/RRF scores
- chunks passed to the model
- answer length
- whether the answer cited book content

---

## External Research Notes

Useful anchors from the quick research pass:

- Supabase hybrid search docs: `https://supabase.com/docs/guides/ai/hybrid-search`
- Anthropic Contextual Retrieval: `https://www.anthropic.com/engineering/contextual-retrieval`
- promptfoo: `https://github.com/promptfoo/promptfoo`
- Ragas: `https://github.com/explodinggradients/ragas`

Treat smaller TypeScript RAG repos as reference implementations, not production dependencies, until their license, activity, API stability, and fit are verified.

Promising ideas to borrow:

- Supabase hybrid search with RRF.
- Anthropic-style contextual chunk headers before embedding and full-text indexing.
- Chapter/section/Q&A-aware chunking rather than fixed character windows.
- Source authority tiers, with book/knowledge-base content boosted over lower-authority sources.
- A promptfoo regression suite for answer quality and citation behavior.

## Practical Next Steps

1. Add a Supabase migration for `book` source support, `metadata jsonb`, `fts`, GIN indexing, and retrieval analytics.
2. Implement `match_content_chunks_hybrid` and update `lib/rag/retrieval.ts`.
3. Build `scripts/ingest-book.ts` with PDF cleanup, structure-aware chunking, contextual headers, embeddings, and idempotent inserts.
4. Update `formatChunksForPrompt` to include chapter, section, page range, chunk ID, and source authority.
5. Update the system prompt to summarize book content and cite book chunks naturally.
6. Add a small promptfoo eval suite for retrieval accuracy, citation grounding, and answer quality.
7. Run a test ingestion against a non-production Supabase project or a clearly labeled staging source before touching production data.